Sensores e Sinais

Revisões sobre a linguagem C e programação de microcontroladores

Luis Silva, Paulo Pedreiras, Pedro Fonseca (DETI/UA/IT)

2025-2026

Informações da aula

- Duração estimada: 2 horas em sala
- Tópico: Introdução ao ESP32-C6-DevKitC-1
- Baseado em: documentação oficial (https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c6/esp32-c6-devkitc-1/index.html#esp32-c6-devkitc-1)
- Material e pré-requisitos
    - Hardware: ESP32-C6-DevKitC-1, cabo USB-C com suporte a dados, 1 LED (qualquer cor), 1 resistência 330Ohm, breadboard e fios de ligação
    - Software: Sistema operativo atualizado (sugerido Linux) e permissões de administrador

Objetivos

- Configurar ambiente de desenvolvimento ESP-IDF no Visual Studio Code
- Compilar, fazer *deploy* (*flash*) e monitorizar um programa no ESP32-C6
- Configurar e manipular entradas e saídas digitais do ESP32-C6

---

1. Ambiente de desenvolvimento (preparação prévia)

O ambiente de desenvolvimento (Figura 1) que nos irá permitir converter o nosso código num *firmware* executável pelo ESP32-C6 consiste em três componentes fundamentais [1]:

- **Toolchain**, um conjunto de compiladores que traduzem o código para instruções nativas do processador do ESP32-C6 (RISC-V).
- **Build tools**, em particular (i) *CMake* para definição de regras (p. ex. o que se pretende construir) e dependências (p. ex. que ficheiros fonte e bibliotecas preciso), e (ii) *Ninja*

para a gestão e execução do processo de compilação de acordo com as especificações CMake.

- **ESP-IDF**, conjunto de bibliotecas base (APIs para configuração e uso de entradas e saídas digitais, Timers, Wi-Fi, etc.), o sistema operativo FreeRTOS modificado para suportar microcontroladores ESP multi-core, e diversos scripts auxiliares para operar a *toolchain*.

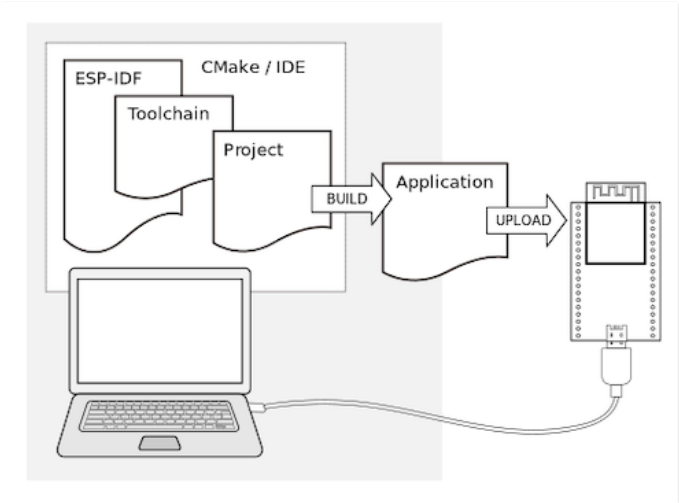

Figura 1 – Componentes principais do ambiente de desenvolvimento [1]

Para além de ser possível utilizar os diversos componentes referidos através de um terminal (linha de comandos), existem alguns *IDE* com extensões que permitem diretamente desenvolver, compilar, gravar (flash) e monitorizar aplicações. Um exemplo é o Visual Studio Code [3] com a extensão *ESP-IDF* [4].

1.1. Instalação de dependências e ESP-IDF

A documentação oficial disponibiliza instruções detalhadas para a instalação dos pré-requisitos, bem como para a instalação de todos os componentes referidos anteriormente através de um único instalador (o ESP-IDF Installation Manager ou EIM), nos diversos sistemas operativos:

- Windows (link) [5]
- Linux (link) [6]
- macOS (link) [7

Nota: Aquando do passo para a instalação dos componentes via o instalador EIM, é aconselhada a escolha da opção "Easy Installation" (Figura 2), que irá instalar todas as ferramentas e configurações em /path/to/user/home/.espressif .

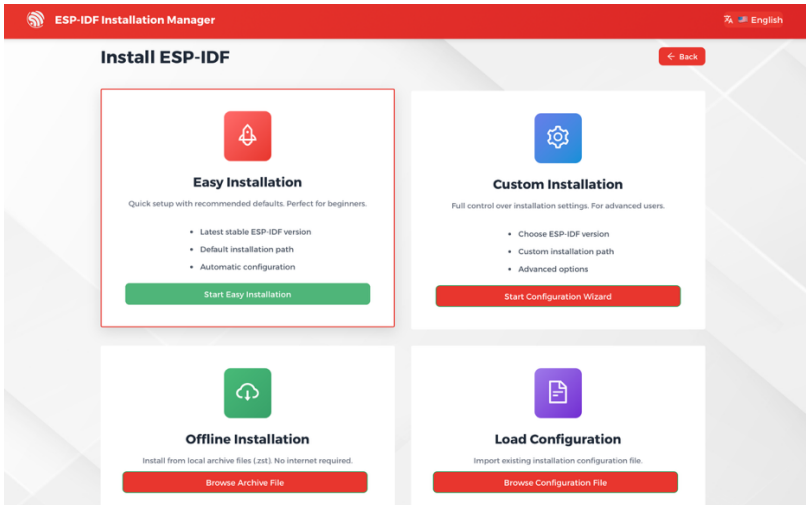

Figura 2 - Menu principal do EIM [7]

1.2. Instalação de IDE (VS Code) e extensão para o ESP-IDF

Para a instalação do IDE VS Code, deverão seguir as instruções oficiais (link) [8] para o respetivo sistema operativo. Nota: nesta fase não é necessária a instalação de componentes ou extensões adicionais como sugerido nos passos 2 e 3 do guia oficial.

Após instalado, a extensão ESP-IDF pode ser encontrada pesquisando **"@id:espressif.esp-idf-extension"** no *marketplace* de extensões do VS Code e instalada de acordo com o ilustrado na Figura 3. Nota: também é possível encontrar a extensão pesquisando pelo seu nome "ESP-IDF". Contudo, a utilização de IDs na pesquisa e instalação de extensões oferece uma maior proteção contra possíveis "imitações" maliciosas.

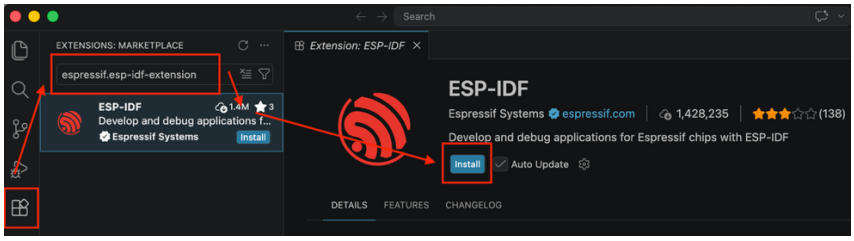

Figura 3 - Instalação extensão ESP-IDF através do marketplace VS Code

Após instalação, feche todas as instâncias do VS Code.

1.3. Criação de workspace e ativação da extensão

Abra uma nova instância do VS Code no diretório de trabalho desejado (p. ex. */path/to/user/home/ss/projetos)*. É aconselhável usar este diretório para conter todos os projetos dado que a extensão irá gerar um ficheiro de configuração necessário à sua ativação neste mesmo diretório. Após aberta a instância no diretório de trabalho, aceda ao menu da extensão ESP-IDF e aceite a ativação da extensão (Figura 4).

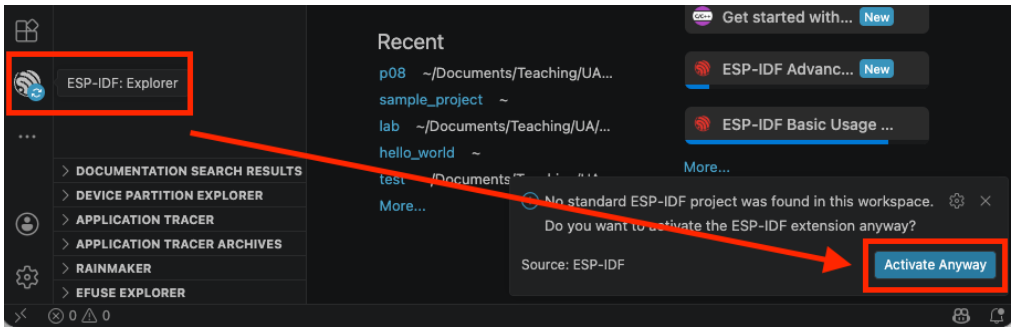

Figura 4 - Acesso ao ESP-IDF explorer e ativação da extensão

Após ativada, será possível visualizar um conjunto de comandos pré-definidos pela extensão que nos irão permitir realizar várias operações tais como criar um projeto, configurar os portos para comunicação com o ESP32-C6, compilar, entre outros.

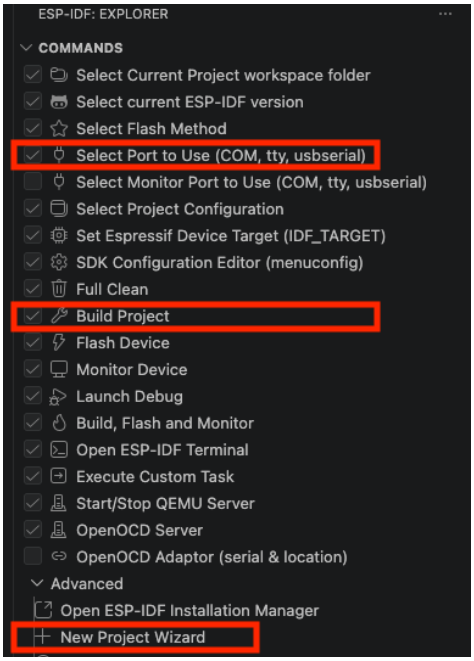

Figura 5 - Menu de comandos da extensão ESP-IDF

2. Novo projeto: “Hello World” (20 min)

Através do menu da extensão ESP-IDF, crie um projeto usando o comando “***New Project Wizard***”. Se existirem ambientes de desenvolvimento bem configurados, irá surgir a opção para selecionar qual destes usar para o projeto. Neste momento deveremos apenas ter uma única opção, a qual deve ser escolhida (Figura 6).

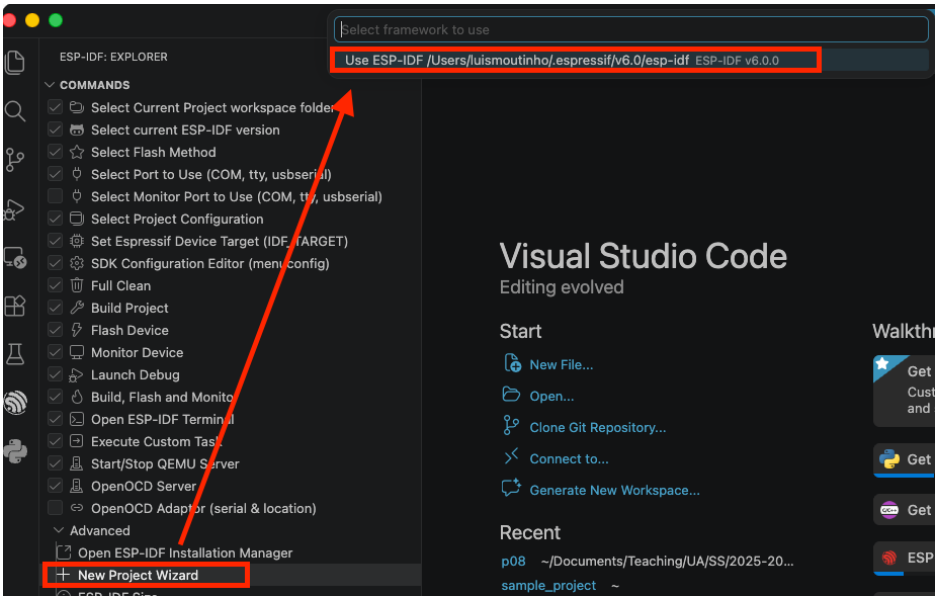

Figura 6 - Escolha da toolchain a usar no novo projeto

Irá aparecer um novo menu de criação de projeto. Aqui podemos escolher criar um projeto com base em exemplos disponibilizados pela ESPRESSIF (“*ESP-IDF Examples*”) ou em *templates* existentes (“ESP-IDF *Templates*”). Selecione o *template* “***sample_project”*** e “**Create Project using template sample_project**” para iniciar o processo de criação da estrutura mínima necessária de um projeto. Irá aparecer uma nova janela para a configuração dos seguintes elementos (Figura 7):

- ***Project name***, o nome para o projeto que deverá ser “***hello_world***”.
- ***Project directory***, o diretório onde guardar todo o conteúdo do projeto. Não é obrigatório, mas é recomendável guardar no *workspace* anteriormente configurado em 1.3.
- ***ESP-IDF Target e ESP-IDF Board***, que especificam o micro e a *board* que serão usados neste projeto. Aqui deverá escolher “**esp32c6**” para o target e “**ESP32-C6 chip (via builtin USB-JTAG)**” para a *board*.

- ***Serial port***, a porta série ligada ao dispositivo. Deixar em "***detect***".
- ***ESP-IDF component directory***, diretório com implementações de módulos (components) a incluir no projeto. Deixar "em branco", não iremos usar esta funcionalidade neste momento.

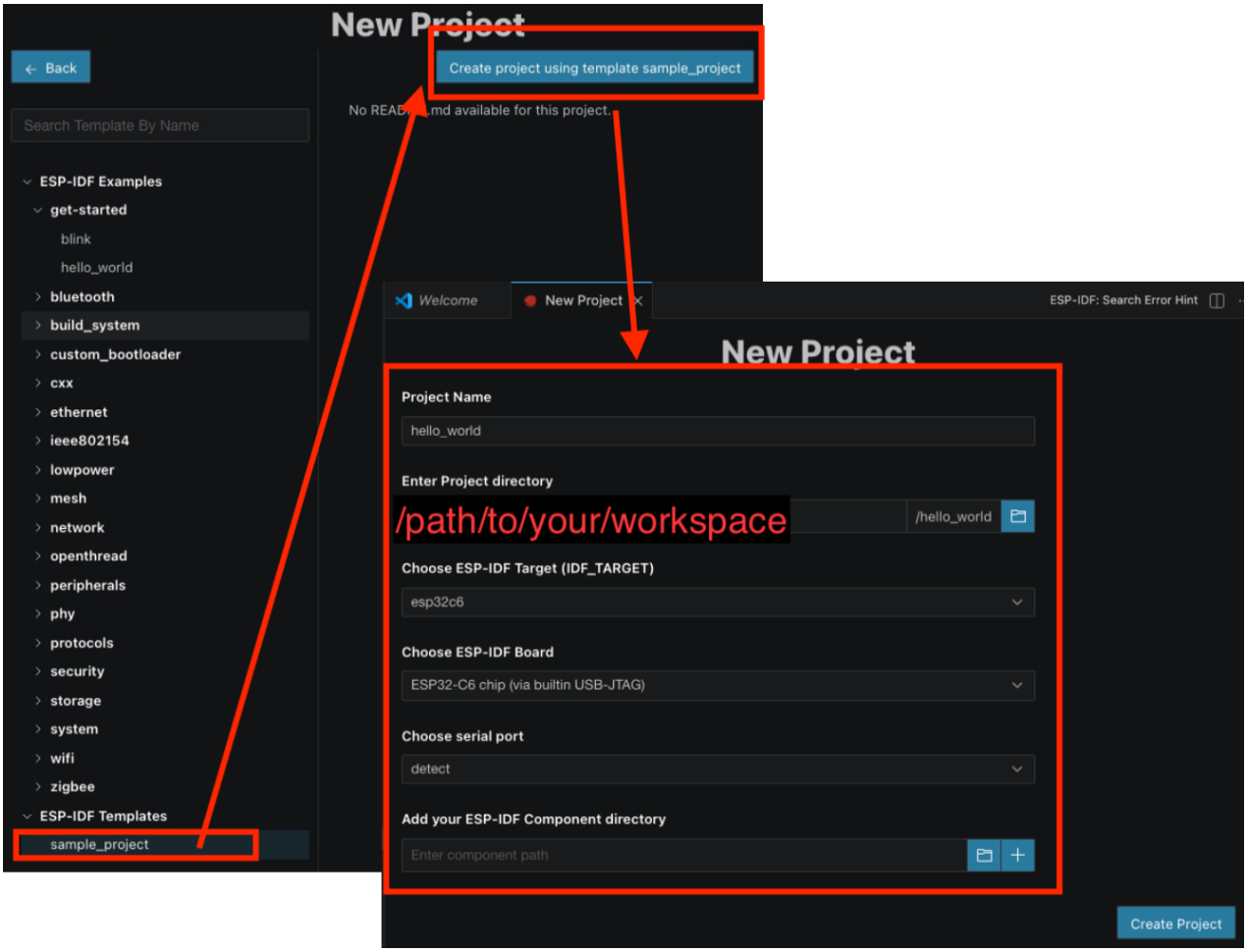

Figura 7 - Criação de um novo projeto com base no sample_project template

Após introdução da configuração inicial, escolha "***Create Project***", espere que a criação deste termine e escolha "**Open Project**" para abrir uma nova instância VS Code para trabalho no novo projeto. Nota: no futuro pode diretamente abrir (ou mudar para) um determinado projeto no VS Code com "***File***" > "***Open folder...***" e escolher o diretório que contém o projeto (não o workspace!).

A Figura 8 mostra a estrutura gerada pela extensão para o projeto hello_world.

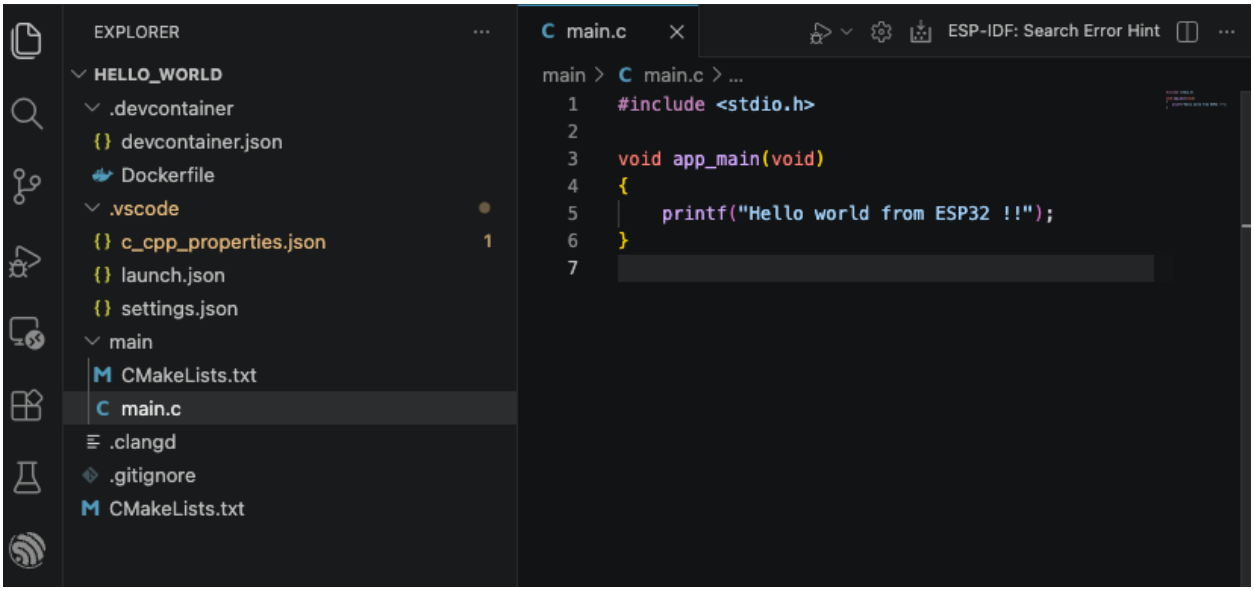

Figura 8 – Estrutura do projeto hello_world

O projeto "esqueleto" contém os seguintes diretórios e ficheiros:

- **.devcontainer**, contém especificações de acordo com o *standard* Development Containers [9]. Estas permitem especificar como criar e utilizar containers (p. ex. Docker) no âmbito do projeto. Por exemplo, é possível especificar um container com todo o ambiente e dependências necessárias para executar uma suite de testes automatizados. Todos os participantes neste projeto teriam automaticamente toda esta funcionalidade sem ser necessário cada um deles criar e configurar o seu próprio ambiente de testes. Este é um tópico avançado e não será abordado nesta altura.
- **.vscode**, contém configurações para diversas utilidades e funções do IDE e extensões. Por exemplo, "*c_cpp_properties.json*" indica à extensão C/C++ onde encontrar os ficheiros necessários para dar *hints* e ajudar no *autocomplete* de funções disponibilizadas pelas bibliotecas do ESP-IDF, "launch.json" que contém as configurações para o *debugger*, e "settings.json" que contém várias definições globais para o VS Code no scope deste projeto que se sobrepõem às definições globais (p. ex., toolchain IDF a utilizar, número de espaços quando se usa a tecla TAB, etc.).
- **main,** contém o código fonte do projeto. Seguindo programação modular, este diretório contém apenas o ficheiro principal "main.c", sendo os restantes módulos implementados num diretório à parte "components".

- **CMakeLists.txt,** um no diretório “main” e outro na raiz do do projeto. Definem a versão mínima do CMake necessária, identificação do projeto, quais as dependências necessárias, e quais os diretórios e ficheiros com código fonte a incluir que são necessários para o projeto.
- **.gitignore,** contém indicações sobre quais os ficheiros a serem ignorados pelas ferramentas de gestão de versão (p. ex. Git).

Vamos agora criar e testar o nosso primeiro programa para o ESP32:

I. Ligue a porta USB-C que se encontra por baixo do botão “Boot” do kit (Figura 9) ao seu computador.
II. Na extensão ESP-IDF, execute o comando “***Select Port to Use***” e escolha “detect” para a extensão tentar descobrir automaticamente qual a porta COM do PC onde está ligado o kit. Se tudo correr bem, irá ver no terminal algo como “Connected to ESP32-C6 on …” bem como alguma informação sobre as features do kit, modo USB e MAC address. <u>Deve executar este comando sempre que ligue o kit ao seu computador pois o porto pode mudar entre sessões.</u>
III. Complete o código fonte no ficheiro ***main.c*** de acordo com a Figura 8. <u>Não se esqueça de gravar o ficheiro com as alterações</u> (normalmente o VS Code guarda automaticamente, mas convém confirmar para evitar possíveis confusões).
IV. Execute o comando ESP-IDF “***Build Project***” para fazer a compilação do código fonte do projeto e gerar os binários que serão posteriormente gravados na memória flash do kit. Se tudo correr bem e não existirem erros no código ou projeto, deverá ver no final do terminal um sumário com o resumo da memória que será utilizada no ESP32.
V. Em seguida execute “***Flash Device***” e escolha a opção “***UART***” para transferir o programa para o kit. Após terminada a operação com sucesso, no terminal verá “Flash Done”.
VI. Finalmente, para monitorizar o programa que está a ser executado no ESP32, execute o comando “***Monitor Device***”. Deverá conseguir observar no terminal as mensagens de inicialização do ESP32 bem como a mensagem “*Hello world from ESP32 !!*” gerada pelo nosso programa.

Parabéns! Tem agora um ambiente funcional para começar a desenvolver ☺ Sempre que necessitar de realizar alterações ao programa, basta repetir os passos IV e V anteriores.

3. Introdução à plataforma ESP32-C6-DevKitC-1 (15 min)

3.1. Características e componentes principais

A Figura 9 apresenta os componentes principais presentes no kit ESP32-C6-DevKitC-1 [10].

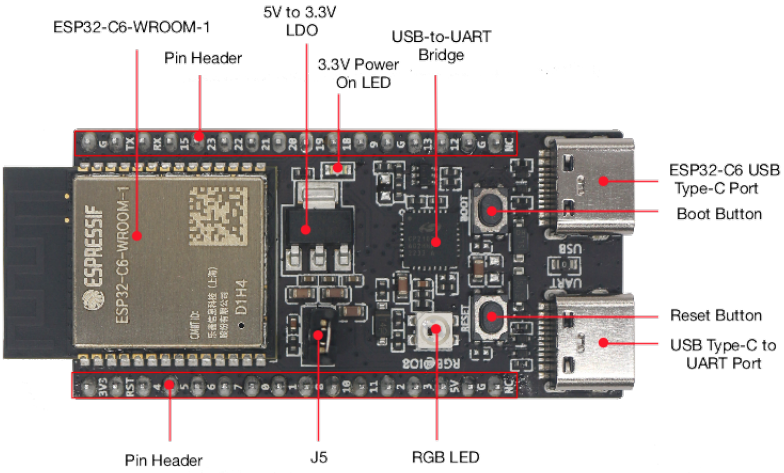

Figura 9 - Vista frontal do kit ESP32-C6-DevKitC-1 [10]

A destacar:

- **Microcontrolador ESP32-C6,** com 8MB de memória Flash e módulos de comunicação sem fios Wi-Fi 6 (apenas banda 2.4GHz), Bluetooth 5 Low Energy, e IEEE 802.15.4 (Zigbee 3.0 e Thread 1.3).
- **Porta USB nativa** (Figura 9, porta superior), ligada diretamente ao microcontrolador, permite alimentar o kit, gravar aplicações na memória Flash, comunicar com o microcontrolador através de protocolos USB/porta-série, e a utilização do standard JTAG para debugging.
- **Pin headers**, permitem o acesso às entradas/saídas de uso geral (General-Purpose Input/Output ou GPIO) do microcontrolador.
- **Botão Reset** (Figura 9, botão inferior), permite reiniciar fisicamente o sistema.
- **Botão Boot** (Figura 9, botão inferior), permite forçar o kit a entrar no modo de programação via porta série. Em modo normal, este pode ser usado pelas aplicações como um botão físico ligado ao *GPIO 9*.
- **LED RGB** (Figura 9, LED inferior), LED multicolor endereçável via *GPIO 8*.

Os GPIOs disponíveis em cada pin header estão ilustrados na Figura 10. De notar que cada um destes GPIOs pode estar associado a uma ou a várias funcionalidades. Por exemplo, o

mesmo GPIO pode ser operado como canal de uma ADC, parte de uma interface I2C, ou simplesmente usado como entrada/saída de uso geral. Cada GPIO tem associada uma função que é utilizada por defeito aquando arranque do ESP32-C6 (p. ex. GPIO usado apenas como entrada de uso geral). Contudo, a função de um determinado GPIO pode ser alterada dinamicamente via software através do uso de funções específicas disponibilizadas pelas bibliotecas ESP-IDF (p. ex., GPIO passa de entrada para saída de uso geral). Para informações mais detalhadas, consultar a documentação oficial em [10].

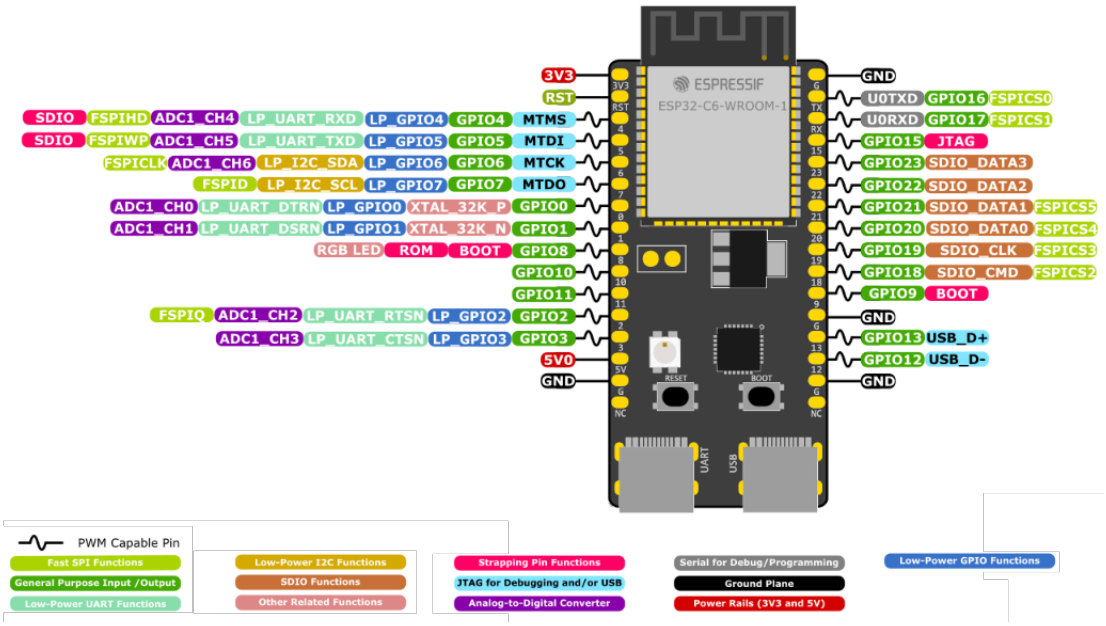

Figura 10 - GPIOS disponíveis nos pin headers do ESP32-C6-DevKitC-1 [10]

3.2. Ecossistema de software (bibliotecas básicas)

O ESP-IDF disponibiliza, organizadas de forma modular, várias bibliotecas e APIs para diversas finalidades. **Para utilizar uma determinada API, é necessário incluir o respetivo cabeçalho (.h) no nosso código fonte, bem como o componente associado nos ficheiros CMakeLists.txt relevantes.** Note que para que uma grande parte das APIs que vai utilizar, tais como GPIO, Timers, ADCs e tasks de RTOS, os respetivos componentes são incluídos por defeito e não é necessária qualquer alteração aos ficheiros CMakeLists.txt.

Segue um resumo das principais APIs que temos disponíveis. **Informação mais detalhada sobre todas as APIs existentes, bem como exemplos do seu uso, pode ser encontrada na documentação oficial em [11].**

3.2.1. API de periféricos (Peripherals API)

Esta API disponibiliza bibliotecas para interagir com o hardware físico do ESP32-C6. Os *headers* para inclusão destas geralmente começam pelo prefixo "***driver/***". Alguns exemplos:

- **GPIO** ("*driver/gpio.h", componente "esp_driver_gpio"*), controlo de entradas e saídas digitais simples.
- **Timers** ("*driver/gptimer.h", componente "esp_driver_gptimer"*), temporizadores de alta precisão por hardware para uso geral (p. ex., interrupções e contagens).
- **Sinais analógicos** ("esp_adc/adc_oneshot.h", componente "esp_adc"), para leituras e conversão de sinais analógicos através de ADCs.
- **Controlador LED e PWM** ("driver/ledc.h", componente "esp_driver_ledc"), controlador principalmente usado para controlo da intensidade de LEDs. Permite também gerar sinais PWM para outros fins.

3.2.2. API de sistema (System API)

API para controlo de diversos recursos e funcionalidades do microcontrolador (p. ex., bootloader, memória heap, power management) e do sistema operativo FreeRTOS. Exemplos:

- **Tasks FreeRTOS** ("freertos/task.h", componente "freertos"), permite criar múltiplas "tarefas" que correm em simultâneo.
- **Logging** ("esp_log.h", componente "log"), permitem controlar o formato das mensagens de log, o nível dos loggers, etc.

3.2.3. API de conectividade e protocolos (Networking API)

API para controlo e interação com os periféricos de comunicações. Exemplos:

- **Wi-Fi** ("esp_wifi.h", componente "esp_wifi"), permite configurar o modo de operação do módulo Wi-Fi (p. ex., como AP ou cliente), modos de seguranças (p. ex. WPA2 ou WPA3), estruturas de dados, etc.

4. Exercícios práticos (60 min)

4.1. Saídas digitais – controlo de um LED

Objetivo: Configurar um pino como saída digital (output) e manipular o seu estado lógico ao longo do tempo, utilizando as bibliotecas de sistema do ESP-IDF.

Material necessário:

- 1 LED (qualquer cor)
- 1 Resistência de 330 Ohm
- Fios de ligação e breadboard
- Placa ESP32-C6-DevKitC-1 e cabo USB-C

Desenvolva um programa que faça o LED externo, ligado ao **GPIO 11**, piscar continuamente a uma frequência exata de **1Hz** e um **duty-cycle de 50%**.

Notas:

- Deverá primeiramente utilizar a função ***gpio_reset_pin()*** para fazer reset à função que o GPIO pudesse ter anteriormente.
- Para configurar o pino como saída digital, use a função ***gpio_set_direction()***.
- Para configurar o nível lógico da saída a qualquer momento, use a função ***gpio_set_level()***.
- Não é necessário o uso de timers por hardware neste exercício, pode explorar o uso de tarefas do FreeRTOS, em particular a função **vTaskDelay()**, para gerar "sleeps" no seu código.

4.2. Entradas digitais – *polling* de estado de um botão

Objetivo: Configurar um pino como entrada digital (input) e ler o seu estado continuamente (técnica de polling).

Material necessário:

- Placa ESP32-C6-DevKitC-1 e cabo USB-C

Desenvolva um programa que imprima a mensagem "[EVENTO] Botão pressionado!" sempre que o botão "Boot" (GPIO 9) integrado na placa seja pressionado. Quando o botão for libertado, deve imprimir "[EVENTO] Botão largado!". As mensagens devem aparecer

apenas uma vez por cada ação, e não continuamente enquanto o botão estiver a ser segurado.

Notas:

- Deverá primeiramente utilizar a função ***gpio_reset_pin()*** para fazer reset à função que o GPIO pudesse ter anteriormente.
- Para configurar o pino como entrada digital, use a função ***gpio_set_direction()***.
- Para ler o nível lógico da entrada a qualquer momento, use a função ***gpio_get_level()***.
- Pesquise pelo conceito de ***Debounce*** e implemente uma solução eficaz.

5. Desafios extra (fora da sala de aula)

5.1. Pisca-pisca com seleção de frequência através de botão

Modifique o código do Exercício 3.3, integrando o botão do Exercício 3.2. Adicione a lógica necessária para que, cada vez que o botão seja pressionado, a frequência com que o LED pisca se altere ciclicamente entre três valores (1Hz, 5Hz e 10Hz).

5.2. *Breathing* LED

Crie um programa que em vez de simplesmente ligar e desligar um LED, faça com que a intensidade luminosa do LED aumente e diminua gradualmente (efeito de "respiração", muito comum em standby de portáteis ou consolas). Dica: explore o conceito de Modulação por Largura de Pulso (PWM) e as capacidades do periférico LEDC (controlador de LED) do ESP32-C6.

Referências

[1] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/get-started/index.html

[2] https://www.freertos.org

[3] https://code.visualstudio.com

[4] https://marketplace.visualstudio.com/items?itemName=espressif.esp-idf-extension

[5] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/get-started/windows-setup.html

[6] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/get-started/linux-setup.html

[7] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/get-started/macos-setup.html

[8] https://code.visualstudio.com/docs/setup/setup-overview

[9] https://containers.dev

[10] https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c6/esp32-c6-devkitc-1/user_guide.html

[11] https://docs.espressif.com/projects/esp-idf/en/latest/esp32c6/api-reference/index.html